

make a documentation based in our app in which i need detail explanation what problem are we solving amongst or differently from existing solution and what dataset are we using and why are we using and about techstack and system workflow in which clearly mention that how our app detects threat how our app will work and how ingest data in our app or how our app monitors the data that is coming in device or system

Here is a professional, high-level technical documentation for CYBER-ASTRA. You can use this for your project report, presentation, or GitHub README.

---

## 📄 CYBER-ASTRA: Technical Documentation

### AI-Based Detection of Cyber Threats in Unidirectional IP Traffic

Problem Statement: SIH26145

#### 1. Executive Summary

CYBER-ASTRA is a professional-grade Security Operations Center (SOC) pipeline designed to detect advanced cyber threats in unidirectional network traffic. Unlike traditional Intrusion Detection Systems (IDS) that rely on bidirectional handshakes, CYBER-ASTRA specializes in "passive monitoring," analyzing one-way traffic flows to identify stealthy threats like C2 beaconing, DNS tunneling, and data exfiltration without ever interacting with the target network.

---

#### 2. The Problem Statement

##### 2.1 The Gap in Existing Solutions

Most existing network security tools (like Firewalls or standard IDS) rely on TCP State Tracking. They expect a "three-way handshake" (Syn  $\rightarrow$  Syn-Ack  $\rightarrow$  Ack). However, in high-security environments, many systems use Unidirectional Gateways (Data Diodes) or stealthy attack vectors where traffic only flows one way.

Existing solutions fail when:

- Traffic is Unidirectional: Standard tools mark one-way traffic as "incomplete" or "errors" and ignore them.
- Encryption is used: When traffic is encrypted (TLS), traditional deep-packet inspection (DPI) cannot see the payload.
- Stealthy C2: Modern malware uses "low-and-slow" beaconing that blends in with normal web traffic.

##### 2.2 The CYBER-ASTRA Difference

CYBER-ASTRA solves this by focusing on Behavioral Heuristics and Traffic Metadata rather than packet content. We don't care what is inside the packet; we care how the packets behave. By analyzing the "shape" of the traffic (timing, volume, and

entropy)), we can detect threats that are invisible to traditional tools.

---

### 3. Dataset Analysis

#### 3.1 Dataset Used

We utilize a specialized Network Flow Dataset (stored as validation\_flows.csv) consisting of unidirectional IP flow records.

#### 3.2 Why this Dataset?

Standard datasets (like KDD99) are outdated. Our dataset includes modern security dimensions:

- Entropy Scores: To detect encrypted tunnels or randomized DGA domains.
- Inter-Arrival Time (IAT): To detect the robotic precision of C2 beaconing.
- TLS Fingerprints (JA4): To identify the specific software (client) making the connection without decrypting the traffic.
- Host/Port Counts: To detect reconnaissance and port scanning.

---

### 4. Technology Stack

| Component             | Technology         | Purpose                                   |
|-----------------------|--------------------|-------------------------------------------|
| Frontend/UI           | Streamlit          | Professional SOC Command Center Dashboard |
| Language              | Python 3.10+       | Core logic and data processing            |
| Data Processing       | Pandas, NumPy      | Vectorized feature engineering and math   |
| AI/LLM interpretation | Qwen3 (via Ollama) | Local, private AI Analyst for threat      |
| Data Model            | Pydantic           | Strict schema validation and type safety  |

|               |               |                                        |
|---------------|---------------|----------------------------------------|
| Visualization | Plotly        | Interactive threat distribution charts |
| Persistence   | JSON (Atomic) | Secure, crash-proof alert storage      |

---

5. System Workflow & Architecture

5.1 Data Ingestion (How data enters the system)

CYBER-ASTRA operates in Passive Monitoring Mode. It does not "sniff" live packets directly to avoid introducing latency or risk. Instead:

- 1. Ingestion Layer: The app ingests flow records (NetFlow/IPFIX style) via `csv_ingest.py`.
- 2. Sanitization: The system cleans the data, handles missing values, and enforces the TrafficFlow schema.
- 3. Passive Nature: The system is strictly read-only. It never sends a packet back to the network, making it invisible to attackers.

5.2 Threat Detection Logic (How it works)

The system uses a Multi-Engine Ensemble Approach. Every flow is passed through 6 specialized engines:

- 1. DDoS Engine: Flags flows with extreme `packets_per_second` and low duration.
- 2. C2 Engine: Uses `inter_arrival_std` to find "heartbeat" patterns typical of botnets.
- 3. DGA Engine: Analyzes `dns_entropy` and `query_length` to find algorithmically generated domains.
- 4. TLS Engine: Matches `tls_fingerprints` against known malicious client patterns.
- 5. Recon Engine: Monitors `dst_port_count` to identify network scanning.
- 6. Exfil Engine: Calculates the `byte_ratio` to detect massive unidirectional data bursts.

Confidence Scoring: Each engine assigns a score (0.0 to 1.0). If the score exceeds a threshold, an alert is generated.

5.3 Monitoring and AI Analysis

Once a threat is detected, the workflow moves to the SOC Analyst Interface:

- 1. Alerting: The threat is saved in the `alerts.json` ledger.
- 2. Visualization: The dashboard displays a real-time feed of alerts.
- 3. AI Interpretation: When an analyst clicks "Analyze," the Qwen3 AI model examines the specific evidence (e.g., "High entropy in DNS query") and provides a

human-readable summary, identifying the "Why" and "How" of the attack.

---

## 6. Summary Table: The Pipeline

Raw Flow Data  $\rightarrow$  Sanitization  $\rightarrow$  Behavioral Feature Extraction  
 $\rightarrow$  Multi-Engine Detection  $\rightarrow$  Atomic Storage  $\rightarrow$  SOC  
Dashboard  $\rightarrow$  LLM Analysis.

\* Churned for 39s • done 4:26 PM